
DESTINY CHURCH TEES VALLEY · SQUARE MEDIA GROUP

iOS & Android Mobile App — Scoping Document

Printable export · Generated 20 July 2026

Destiny Church Mobile App — iOS & Android Scoping Document

Status: Draft for review

Date: 19 July 2026

Author: Square Media Group (prepared with Claude Code)

This document scopes a native iOS and Android app for Destiny Church Tees Valley, based on a full review of the existing `destinychurch` repository. It covers what already exists and is reusable, the proposed architecture, the Matrix-based group chat design with its safeguarding constraints, ChurchSuite integration, payments, the sermon/podcast feed, and the open decisions that must be resolved before build.

1. Where we are today (repo audit)

The current repository is a **Next.js 16 (App Router) web platform** deployed on Vercel, with Supabase (PostgreSQL + Auth + Storage), Resend email, Stripe (shop only), YouTube Data API v3 for sermons, and Buzzsprout RSS for the podcast. There is **no mobile app code, no chat/messaging code, and no ChurchSuite API integration** anywhere in the repo today — ChurchSuite is consumed entirely through public embeds and iframes.

What exists that is directly relevant to the app:

Area	What exists	File(s)	Reusable for the app?
Events	ChurchSuite public calendar JSON embed (<code>destinytees.churchsuite.com/embed/calendar/json</code>), rendered natively	<code>app/whats-on/page.tsx</code> , <code>components/whats-on/EventsGrid.tsx</code> , <code>components/home/WhatsOnSection.tsx</code>	Yes — proves the calendar JSON feed works without the full API; a good fallback layer
Giving	ChurchSuite donate page in an iframe (<code>/donate</code>)	<code>app/give/page.tsx</code> , <code>components/give/GiveCTA.tsx</code>	Yes — same embed goes into a WebView
Forms	~10 ChurchSuite forms embedded via a shared iframe wrapper with a loading state	<code>components/ChurchSuiteEmbed.tsx</code> , used on <code>connect/</code> , <code>baptism/</code> , <code>contact/</code> , <code>serve/</code> , <code>new-here/</code> , <code>connect-card/</code> , <code>child-dedication/</code> , kids camp, etc.	Yes — pattern maps 1:1 to a WebView screen
Sermons (video)	YouTube Data API v3 fetch + thumbnail proxy (<code>/api/youtube/thumbnail/[id]</code>), series/speaker filtering, 50+ videos	<code>lib/youtube.ts</code> , <code>lib/youtubeIframe.ts</code> , <code>lib/collections.ts</code> , <code>app/sermons/</code>	Yes — the server-side fetch/caching logic becomes app API endpoints
Podcast	Buzzsprout RSS parsed server-side into typed episodes (title/speaker parsing, artwork, duration, MP3 URL), with a web audio player	<code>lib/podcast.ts</code> , <code>components/sermons/podcast/*</code>	Yes — same parsed feed served to the app; player UI rebuilt natively
Auth	Supabase Auth (email/password + OTP/OAuth callbacks), used for staff/admin only ; middleware-gated admin area; login rate limiting	<code>middleware.ts</code> , <code>lib/supabase*.ts</code> , <code>lib/loginRateLimit.ts</code> , <code>app/login/</code> , <code>app/auth/</code>	Partially — infrastructure and patterns reusable, but the app needs member identity, which does not exist today
Content	Posts, training, courses, banners, popups in Supabase with admin CRUD	<code>lib/posts.ts</code> , <code>lib/training.ts</code> , <code>app/admin/*</code>	Optional — could surface training/content in-app later; out of MVP scope
Safeguarding	Public safeguarding policy page	<code>app/safeguarding/page.tsx</code>	Policy text must be reviewed against the chat permission model (see §8)

Notably, `app/help/page.tsx` currently tells users: "A dedicated Destiny app is not currently available." — that copy will need updating at launch.

Key gap: there is no member-facing identity system, no ChurchSuite API client, no chat infrastructure, and no push notification capability. Those are the four genuinely new builds.

2. Recommended overall architecture

```
iOS / Android app (single codebase)
├── App API / BFF (backend-for-frontend)
│   ├── ChurchSuite API client (groups, events, forms, giving links)
│   ├── Sermon feed (YouTube API, reusing lib/youtube.ts logic)
│   ├── Podcast feed (Buzzsprout RSS, reusing lib/podcast.ts logic)
│   ├── Member auth + directory sync (adult/minor classification)
│   └── Safeguarding enforcement service (Matrix admin operations)
├── Matrix homeserver (self-hosted Synapse, UK/EU)
│   └── Custom Synapse module enforcing chat rules (§4)
└── WebViews → ChurchSuite hosted checkout, forms, donate page
```

2.1 App framework: React Native (Expo)

Recommendation: **React Native with Expo**, one codebase for iOS and Android.

- The existing team ships TypeScript + React 19 + Zustand daily; React Native reuses that entire skillset, plus shared types and utility code from this repo (`lib/youtube.ts` types, `lib/podcast.ts` types, zod schemas).
- The app is content + chat + WebView — nothing (AR, heavy graphics, custom Bluetooth) that would justify two native codebases in Swift/Kotlin.
- Matrix has a maintained JS SDK (`matrix-js-sdk`) that works in React Native; since we are **not** using E2EE (§4), we avoid the hardest part of Matrix-on-RN (the crypto/rust bindings), making the JS SDK path significantly simpler. Alternatively the app can talk to Matrix purely via its plain HTTP client-server API through our BFF.
- Expo gives us OTA updates (within store rules), managed builds (EAS), and push notification plumbing.

Flutter is the main alternative (Element X-adjacent tooling exists), but it forfeits all code/skill reuse from this repo. Not recommended.

2.2 App API / BFF

A thin backend-for-frontend, hosted alongside (or inside) the existing Next.js app as API routes — this repo already runs serverless API routes on Vercel, and the sermon/podcast fetch logic already lives server-side here. The BFF:

- Holds all secrets (ChurchSuite API credentials, YouTube key, Matrix admin token). **The app never talks to ChurchSuite's API directly.**
- Normalises ChurchSuite responses into stable app-facing shapes, so ChurchSuite API changes don't force app releases.
- Enforces the "no local caching that could drift" rule: responses are proxied live, with only short-TTL in-memory/edge caching (seconds-minutes) for rate-limit protection — never persisted app-side as a shadow database. On ChurchSuite outage, the BFF returns an explicit "temporarily unavailable" status and the app shows a graceful degraded state per surface (see §5.3).

One caveat on hosting: the safeguarding enforcement service (§4.4) needs to react to Matrix events reliably and may suit a small always-on service colocated with the homeserver rather than Vercel serverless. Decide at technical design stage.

3. Identity, membership, and the adult/minor boundary

Everything in the safeguarding model depends on knowing **who a user is and whether they are 18+**, so this is foundational.

- **ChurchSuite is the system of record** for people. The BFF syncs (or live-looks-up) contacts/children via the ChurchSuite API; date of birth drives the adult/minor classification. A user with no DOB on record is treated as a **minor by default** (fail-safe).
- **App accounts are provisioned, not self-signed-up.** Sign-in should verify the person against their ChurchSuite record (invite links / email verification against the address book). Reuse Supabase Auth — already integrated in this repo — as the credential layer, with a mapping table `app_user → churchsuite_contact_id → matrix_user_id`.
- **No phone numbers anywhere:** no phone sign-up, no phone verification, phone fields never displayed in any profile or member list in-app. This is what closes the off-platform side-channel.
- **Role model** (sourced from ChurchSuite tags/roles, mirrored into Matrix power levels):
 - `senior_leadership` — can create groups, safeguarding visibility
 - `group_leader` (small group / connect group leaders) — can create groups
 - `member_adult` — join groups, chat
 - `member_minor` — join permitted groups, chat
 - `safeguarding_officer` — receives alerts, has review access (§4.5)

- **Age transitions:** a nightly job re-evaluates DOBs. When a minor turns 18 their classification flips, which can *satisfy* the 2-adult rule but never break it, so this is low-risk — but it must be automated, not manual.

4. Group chat — self-hosted Matrix

4.1 Homeserver choice: Synapse

Recommendation: Synapse, not Dendrite or Conduit, because every safeguarding rule below is enforced server-side via **Synapse's module API** (spam-checker / third-party-rules callbacks), which Dendrite and Conduit do not offer in mature form. Dendrite/Conduit would force enforcement into a proxy layer in front of the homeserver — more moving parts, weaker guarantees. Synapse is the reference implementation, battle-tested, and fine at church scale (hundreds of users) on a single modest server with PostgreSQL.

Run it **closed-federation** (`federation_domain_whitelist: []` / federation disabled entirely): our users can only talk to our users. Registration disabled — accounts are provisioned by the BFF via the admin API when an app account is created.

4.2 The four hard rules and how each is enforced

All rules are enforced **server-side in a custom Synapse module** — never only in the app UI. A modified client must not be able to bypass any of them.

1. **No 1:1 messaging for anyone (minors or adults).**
2. Module rejects any room creation where `is_direct: true` or `preset: trusted_private_chat`.
3. Module rejects any invite/join that would result in a room with fewer than 3 human members (this also prevents "fake groups" of 2 used as DMs).
4. Client UI has no DM affordance at all; the server rule is the backstop.
 1. **Group creation restricted to senior leadership and small group leaders.**
 2. Module's `on_create_room` callback checks the creator's role (sync'd from ChurchSuite into a lookup the module can read — a small DB table or the BFF's API).
 3. Everyone else gets `M_FORBIDDEN` on room creation. In practice, group creation will normally flow through the BFF anyway (create room + set power levels + invite members atomically), but the module rule means even a raw Matrix API call can't bypass it.
1. **Every group must have ≥ 2 adult (18+) members, at creation and continuously.**
2. At creation: BFF refuses to create the room until at least 2 verified adults are in the initial member list; the Synapse module double-checks.
3. On membership change: module intercepts every leave/kick/ban/deactivation. If the event would drop the adult count below 2, the configured drop-below-2 behaviour fires (see §8, decision D1 — block the leave, freeze the room, or notify safeguarding; my recommendation is **freeze + notify**, detailed there).
4. "Adult" is determined from the ChurchSuite-sync'd DOB, not self-declared.
1. **No E2EE — encrypted at rest, fully reviewable.**
2. Synapse config: `encryption_enabled_by_default_for_room_type: off`; module rejects any `m.room.encryption` state event so no client can switch encryption on.
3. At-rest encryption via full-disk/volume encryption on the homeserver + encrypted PostgreSQL storage and encrypted backups.
4. Because messages are plaintext to the server, the safeguarding review capability (§4.5) works after the fact — the whole point of this trade-off.

4.3 Other Matrix configuration

- **Media:** uploads stored on the homeserver (encrypted volume), size-capped; consider restricting to images/PDF initially. Media is subject to the same review access as messages.
- **Retention:** define a retention policy (e.g. retain everything for N years per safeguarding policy — this is a policy decision, see §8/D6). Do **not** enable Matrix per-room self-destructing message retention.
- **Redactions:** users can redact ("delete") their own messages in-client, but Synapse should be configured to retain redacted content for the safeguarding review window rather than purging it.
- **Display identity:** display names locked to the real name from ChurchSuite (module rejects displayname changes, or BFF resets them) — no pseudonyms in a safeguarding context.

4.4 Safeguarding enforcement service

A small companion service (or part of the BFF) that:

- Provisions/deactivates Matrix accounts as app accounts change.
- Syncs role and adult/minor status from ChurchSuite into the lookup the Synapse module reads.
- Runs the group-creation flow (room + power levels + membership) atomically.
- Receives module webhooks (adult-count breach, flagged events) and routes notifications to the safeguarding role.

- Nightly reconciliation: re-checks every room still satisfies all invariants (defence in depth against any missed edge case).

4.5 Reviewability

- A minimal internal review tool (could live in the existing `/admin` area of this repo, gated to `safeguarding_officer` + senior leadership) that can pull a room's full history via the Synapse admin API for incident investigation.
- Every review access is itself **audit-logged** (who viewed what, when) — reviewability must not become unsupervised browsing of members' conversations. This should be written into the safeguarding policy alongside the technical build.
- App onboarding and terms must state clearly that chats are not end-to-end encrypted and are reviewable by the safeguarding team. This transparency is both ethically necessary and a GDPR requirement (lawful basis + privacy notice update — the existing `privacy-policy.txt` / `app/privacy` content does not cover any of this yet).

5. ChurchSuite integration

5.1 Principle

ChurchSuite remains the **system of record** for people, groups, events, forms, and giving. The app is a presentation layer. The BFF talks to the ChurchSuite API (read/write); the app talks only to the BFF.

5.2 Surfaces

Surface	Source	App treatment
Connect Groups	ChurchSuite Smallgroups module (API)	Native list/detail; membership data also seeds Matrix room membership suggestions. Group <i>signup</i> can write via API or fall back to the existing embed (<code>/forms/twuneil</code> pattern already on <code>app/connect/page.tsx</code>)
Events	ChurchSuite Calendar (API; public JSON embed already proven in <code>app/whats-on/page.tsx</code> as fallback)	Native list/detail with images (<code>cdn.churchsuite.com</code> already whitelisted in <code>next.config.ts</code>); "sign up / buy tickets" opens ChurchSuite event page in WebView
Forms	ChurchSuite hosted forms (the ~10 already embedded across the site)	WebView screens reusing the exact URLs from the web components; native screen chrome around them
Giving	ChurchSuite <code>/donate</code> hosted flow	WebView (see §6)

5.3 Graceful degradation (no drift-prone caching)

Per the agreed constraint, the app must not build a local cache that can drift from ChurchSuite:

- BFF proxies live, with only short-TTL ($\leq$ a few minutes) non-persistent caching for rate-limit smoothing.
- If ChurchSuite is unreachable: events/groups screens show a clear "temporarily unavailable — try again shortly" state (with any short-TTL data shown as explicitly stale if present); forms/giving WebViews show ChurchSuite's own error.
- **Chat is unaffected** by ChurchSuite outages: Matrix membership/roles are synced state, and an API blip must never freeze rooms or fire safeguarding alerts. Sync jobs treat "ChurchSuite unreachable" as "no change", never as "member data gone".

5.4 Pre-build checks (blocking)

1. **API scope on Destiny's plan:** confirm the ChurchSuite API on Destiny's current plan exposes *read and the required write* access for Smallgroups, Calendar, Address Book (including children/DOB — essential for §3), and Forms. (Open item D3.)
1. **Ask ChurchSuite directly** whether their hosted checkout/donate embed has recently passed App Store / Play Store review inside other church clients' apps (they will know; several UK churches ship apps on ChurchSuite). (Open item D5.)

6. Payments

No native payment build. Both flows are ChurchSuite's existing hosted checkout in a WebView; PCI DSS, Gift Aid, fraud, and reconciliation all remain inside ChurchSuite, exactly as on the website today (`app/give/page.tsx` embeds `/donate` already).

- **Giving (tithes/offerings):** Apple's rules (App Review Guideline 3.2.2 area) permit approved nonprofits to collect charitable donations outside IAP; Google Play has an equivalent charitable-giving carve-out. Destiny is a registered charity — this is the standard church-app pattern.
- **Event/ticket payments (camps, conferences):** tickets for real-world events are **physical goods/services**, which have always been outside IAP scope on both stores (this is how Eventbrite etc. work) — the post-Epic external-payment loosening is helpful margin, not the primary basis. Still, because the Epic ruling is US case law and Destiny's storefront is the **UK**, one direct confirmation is warranted (open item D5) — and the ChurchSuite question in §5.4(2) largely answers it empirically.

- The existing Stripe integration in this repo (`lib/stripe.ts` , `/shop`) is **out of scope** for the app MVP; if the shop is ever added, physical merch is likewise outside IAP.

7. Sermon & podcast feed

Lowest-risk part of the build — the server side already exists in this repo.

- **Video:** expose the existing YouTube fetch logic (`lib/youtube.ts` — channel uploads, view counts, durations, series/speaker grouping via `lib/collections.ts`) as BFF endpoints; the thumbnail proxy (`/api/youtube/thumbnail/[id]`) is reused as-is. Playback in-app uses the official YouTube embedded player (react-native-youtube-iframe) — **required** by YouTube ToS since we're not rehosting; background/audio-only playback of YouTube content is not permitted, which is exactly what the podcast covers.
- **Podcast:** `lib/podcast.ts` already parses the Buzzsprout RSS (`feeds.buzzsprout.com/268765.rss`) into typed episodes with speaker parsing, artwork, and direct MP3 URLs. Serve that through the BFF; build a **native audio player** (expo-audio / react-native-track-player) with background playback, lock-screen controls, and scrubbing — a genuinely better experience than the web player and the main "native feel" win of this surface.
- No rehosting, no publishing pipeline changes, no historical migration — the feed simply reflects what's already published to YouTube and Buzzsprout.
- **Livestream:** the site has `app/live/` ; surfacing the livestream in the same feed tab is near-free and worth including in MVP.

8. Safeguarding summary and open decisions

How the requirements are met structurally

Requirement	Mechanism
No adult–minor 1:1 contact	No 1:1 for anyone , enforced server-side (Synapse module), not just by policy or UI
Minimum 2 adults per group	Enforced at creation (BFF + module) and on every membership change (module)
Only vetted leaders create groups	Role-gated room creation, roles sourced from ChurchSuite
Incidents investigable	No E2EE; encrypted at rest; audited review tool via Synapse admin API
No off-platform side-channel	No phone numbers collected, verified, or displayed anywhere in the app

Decisions required before build (owners: Destiny leadership + safeguarding lead unless noted)

- **D1 — Behaviour when a group drops below 2 adults.** Options: (a) block the leave action, (b) freeze the chat, (c) notify a safeguarding role. **Recommendation: (b)+(c)** — freeze the room read-only the moment adult count < 2 and notify safeguarding, auto-unfreezing when a second adult (re)joins. Blocking the leave (a) alone is problematic: an adult must always be able to leave (they may have safety reasons of their own, and deactivations/emergencies can't be "blocked"), so a block-based rule always has holes; freeze+notify fails safe in every path.
- **D2 — Hosting target for the Matrix homeserver.** Own infrastructure vs a UK/EU managed host. Given a small team and safeguarding-critical uptime/backups, a **UK/EU managed VPS or managed-Matrix provider with a data-processing agreement** is the pragmatic recommendation; data residency must be UK/EU for GDPR either way. (Square Media Group to recommend; Destiny to sign off.)
- **D3 — ChurchSuite API scope check** on Destiny's current plan: confirm write access and children/DOB read access (blocking for §3 and §5).
- **D4 — Push notification provider and data residency.** Practical reality: APNs (Apple) and FCM (Google) are unavoidable for reliable delivery on their platforms. Mitigation: send **content-free notifications** ("New message in your group") so no message content or minor's data transits US-controlled infrastructure; document APNs/FCM as processors in the privacy notice. Signal (Matrix's push gateway) self-hosted in the UK/EU sits in front of both.
- **D5 — UK storefront confirmation** on external payment embeds (Apple, post-Epic) + ask ChurchSuite whether their checkout embed has passed App Store/Play review for other church clients recently. (Square Media Group to action; cheap, do first.)
- **D6 — Safeguarding policy review against the permission model.** The written policy (see `app/safeguarding/page.tsx`) must be checked against §3/§4 — including message retention duration, who holds the `safeguarding_officer` role, and the review-access audit process — **before any chat permission code is written**. This is a stated hard gate.

9. Suggested phasing

Phase 0 — Confirmations (1–2 weeks, mostly waiting on third parties)

D3 (ChurchSuite API scope), D5 (payments/storefront), D2 (hosting decision), D6 kickoff (safeguarding policy review). No code beyond spikes.

Phase 1 — App shell + content (read-mostly, low risk)

Expo app skeleton, design system from the DC brand guide (PDF in repo root), BFF with sermon/podcast/events endpoints (porting `lib/youtube.ts`, `lib/podcast.ts`, calendar JSON logic), native podcast player, YouTube player, events list/detail, WebView screens for giving + forms. **This phase is shippable on its own** as a v1 without chat, and exercises the store-review payment question early with the lowest-risk build.

Phase 2 — Identity

Member provisioning against ChurchSuite address book, Supabase Auth integration, adult/minor classification, role sync, privacy notice updates.

Phase 3 — Chat (gated on D1 + D6 sign-off)

Synapse deployment (closed federation, no E2EE, encrypted at rest), custom enforcement module (the four rules), safeguarding service + nightly reconciliation, group chat UI, push via Sygnal – APNs/FCM (content-free), safeguarding review tool in `/admin`, audit logging.

Phase 4 — Hardening + launch

Penetration/abuse testing of the chat rules specifically (attempt to create DMs, sub-2-adult rooms, enable encryption, change display names via raw API), load test, store submission, staged rollout, update `app/help/page.tsx` copy and `REPOSITORY_DOCUMENTATION.md`.

10. Principal risks

1. **ChurchSuite API gaps** (write access, children/DOB, forms) — why D3 is Phase 0. Fallbacks exist for every surface (embeds/WebViews, calendar JSON) except identity: if DOB isn't API-readable, the adult/minor model needs a manual verified import process, which is workable but operationally heavier.
1. **App Store review of the payment WebViews** — mitigated by the nonprofit/physical-goods framing, D5 confirmation, and shipping Phase 1 first so any review friction surfaces before chat is at stake.
1. **Enforcement correctness** — the 2-adult and no-1:1 rules are only as good as the Synapse module; hence server-side enforcement (never UI-only), nightly reconciliation, and a dedicated abuse-testing pass in Phase 4.
1. **Operational ownership of the homeserver** — someone must own patching, backups, and monitoring long-term; feeds the D2 decision.
1. **GDPR surface grows substantially** (children's data, reviewable private messages, new processors) — privacy notice, DPIA-style assessment, and retention policy work should run alongside Phase 2/3, not after.

Appendix A — ChurchSuite API v2 technical reference

Sourced directly from the OpenAPI 3.0 specifications in github.com/ChurchSuite/churchsuite-api (`src/*.yaml`, spec version 2.89.2). This supersedes the general statements in §5 with concrete endpoint- and field-level detail for build planning. **v1 is deprecated and expected to be discontinued in 2027 — build against v2 only.**

A.1 Authentication & account model

- **Base URL:** `https://api.churchsuite.com/v2` — a single global endpoint; the ChurchSuite **account is identified by the credential itself** (API key or OAuth token), not by a subdomain in the URL. The BFF holds one credential per account (Destiny's) and never needs per-user ChurchSuite auth.
- **Two auth schemes**, usable per-endpoint:
 - `api_enabled_user` — an API key sent as an `Authorization` header (apiKey-style).
 - `oauth_app` — OAuth2 **client-credentials** flow (app-to-app, no user login step) against `https://api.churchsuite.com/v2/oauth/token`. This is the right fit for our BFF: it's a server-to-server integration, not a "sign in with ChurchSuite" flow for members.
- **Scopes are granular per module and per read/write**, e.g. `addressbook.read`, `addressbook.write`, `children.read`, `children.write`, `smallgroups.read`, `smallgroups.write`, `calendar.read`, `calendar.write`, `giving.read`, `giving.write`, `bookings.read`, `bookings.write`, plus account-level `profile.read`, `account`, `brands.read`, `users.read`, `usergroups.read`, or the all-encompassing `full_access`. **Recommendation: request the narrowest scope set the app actually needs** (e.g.

`addressbook.read` , `children.read` , `smallgroups.read` + `.write` , `calendar.read` , `giving.read`)
rather than `full_access` — this is the concrete list to put in front of ChurchSuite for the D3 access-scope conversation.

- **Pagination:** `page` / `per_page` query params on all list endpoints (`per_page` capped at 250 on the modules checked); responses return a `data` array plus a `pagination` object.
- **Rate limiting:** the spec documents a `429 Too Many Requests` response; exact limits/headers aren't published in the spec itself — confirm numeric limits directly with ChurchSuite during Phase 0 so the BFF's short-TTL caching (§5.3) is sized correctly.

A.2 Modules and what they mean for each app surface

Module (<code>src/*.yaml</code>)	Relevant endpoints	What it gives the app
<code>addressbook.yaml</code>	<code>GET/POST /addressbook/contacts , /{id} , /notes , /tags , /key_dates</code>	Adult contact records. Contact schema includes <code>date_of_birth</code> (optional field — confirms the "no DOB → treat as minor" fail-safe in S3 is necessary, not paranoid), <code>email</code> , <code>mobile</code> / <code>telephone</code> (never surfaced in-app per the no-phone-numbers rule), <code>spouse_id</code> , <code>communication</code> consent flags, <code>privacy</code> visibility flags, and a <code>status</code> (active/archived/pending) — useful for filtering out archived contacts from group-eligibility checks. No parent/child linkage lives here — that's a separate module.
<code>children.yaml</code>	<code>GET/POST /children/children , /parent_carer_relationships , /key_dates , /tags</code>	Minor records, held deliberately separate from adult contacts. Confirms the DOB-based adult/minor split in S3 maps directly onto ChurchSuite's own data model (Addressbook = adults, Children = minors, linked via <code>/children/parent_carer_relationships</code>). Also carries <code>photo_video_consent</code> (internal/external) and <code>additional_needs</code> / <code>medical</code> fields — not needed by the app , but their presence is a reminder that any BFF sync job touching this module must scope its ChurchSuite request tightly (i.e. never pull the full child record when only DOB + parent/carers linkage is needed) to avoid over-fetching safeguarding-sensitive medical data into app infrastructure.
<code>smallgroups.yaml</code>	<code>GET /smallgroups/groups , GET/POST/PUT/DELETE /smallgroups/members , /roles</code>	Connect Groups. <code>Member.person</code> is a tagged union of <code>{type: contact</code> child} Church model allow group which the c adult guard my_ex group bool natur group. driv: creat match room a signup useful group app.
<code>calendar.yaml</code>	<code>GET /calendar/events , /invites , /signups , /tickets</code>	Events. Confirms the public JSON embed already used on <code>/whats-on</code> (§1) has a fuller authenticated equivalent; <code>signup_options</code> carries <code>capacity</code> , <code>allow_cancel</code> , <code>confirmation_email</code> — richer than the public embed. No payment fields on the Event object itself (see Bookings, below).
<code>bookings.yaml</code>	<code>GET /bookings/bookings , /charges , /prices , /types</code>	Paid events/camps. <code>Charge.payment_method</code> (api/bank/card/cash/cheque) and <code>Price.amount</code> / <code>time_unit</code> exist for <i>reporting</i> , but the spec exposes no endpoint to create a checkout session or payment URL — payment initiation only happens through ChurchSuite's own

Module (<code>src/*.yaml</code>)	Relevant endpoints	What it gives the app
		hosted flow. This is a direct, spec-level confirmation of §6: there is no API path to a native/custom checkout even if we wanted one — the WebView-to-hosted-checkout approach in §6 isn't just the sensible choice, it's the only one the API supports.
<code>giving.yaml</code>	<code>GET /giving/donations , /funds , /givers , /pledges , /declarations</code>	Giving. <code>Donation.giftaid</code> (UK Gift Aid claim tracking) and <code>Declaration</code> (Gift Aid declarations) are read-only via the API — Gift Aid administration stays entirely in ChurchSuite as required. <code>Fund.visible_in_donate / donate_frequency</code> (oneoff/oneoff_recurring/recurring) confirm funds are configured for the hosted Donate page we already embed; again, no donation-creation endpoint exists — same conclusion as Bookings, reinforcing §6's "no native payment rebuild." <code>Account.integrations.stripe.accounts</code> (<code>GET /account/integrations/stripe/accounts</code>) confirms ChurchSuite's payment processing sits on Stripe underneath its own hosted checkout.
<code>account.yaml</code>	<code>GET /account , /brands , /sites , /users , /user_groups</code>	Multi-site (<code>site_ids / all_sites</code> appear throughout every module) — relevant only if Destiny is or becomes multi-site; otherwise ignorable for MVP. <code>GET /account/users + /user_groups</code> is a plausible source for the <code>senior_leadership</code> role mapping in §3, worth checking against however Destiny's staff/leaders are actually tagged today.
<code>attendance.yaml , planning.yaml , rotas.yaml , network.yaml</code>	—	Rota/service-planning/multi-church-network modules — out of scope , no app surface calls for them.

A.3 What this changes in the plan

- **D3 (API scope check) can now be a specific ask, not a vague one:** request `addressbook.read` , `children.read` , `smallgroups.read + write` , `calendar.read` , `giving.read` (and `bookings.read` if in-app booking status is wanted) on Destiny's plan, via either an API key or an OAuth client-credentials app — confirm ChurchSuite offers OAuth client-credentials apps on Destiny's plan tier, not just static API keys.
- **The "no native payment rebuild" decision in §6 is now spec-confirmed, not just policy-preferred** — Bookings and Giving expose no payment-initiation endpoints at all, so the WebView-to-hosted-checkout approach is the only technically possible one, which resolves any residual "could we integrate more tightly later" question.
- **The adult/minor data split in §3 matches ChurchSuite's own Addressbook/Children module boundary** — the BFF's role-sync job should read `date_of_birth` from Addressbook for members 18+ and treat anyone who only exists in the Children module (or has no DOB in Addressbook) as a minor by default, per the fail-safe already specified.
- **Over-fetching risk:** the Children module schema carries medical/safeguarding fields the app has no legitimate use for; the BFF's ChurchSuite client should request field-limited responses where the API supports it, or immediately discard unused fields server-side, so no medical data is retained in app infrastructure it doesn't belong in.

Appendix B — Apple Human Interface Guidelines considerations

Apple's HIG (developer.apple.com/design/human-interface-guidelines) is a large, JavaScript-rendered reference rather than a fixed document, so this appendix summarises the **stable, well-established guidance areas** relevant to this app's specific surfaces, to brief against design and to revisit page-by-page at high-fidelity design time rather than treat as exhaustive or verbatim-quoted.

B.1 Navigation shell

- **Tab bar:** Apple's guidance caps practical usage at **5 visible tabs**; this app's four content pillars (Sermons/Podcast, Events, Groups/Chat, Give) plus a Home/More tab fits comfortably. Use SF Symbols for tab icons for free dark-mode/Dynamic Type/accessibility behaviour rather than custom icon assets. Avoid overloading the tab bar with actions that belong on a screen instead (e.g. "New Group" is a leader-only in-context button, not a tab).
- **Modality:** ChurchSuite WebView screens (forms, giving, event signup) should generally push onto the navigation stack like any other screen, not present as a sheet/modal, so the back gesture and nav bar back button behave predictably — reserve modal presentation for short, self-contained tasks the user explicitly opts into and expects to dismiss (e.g. a confirmation flow), not for a form that's really "content" in the app's structure.

B.2 WebViews (ChurchSuite embeds)

- Use `WKWebView` (via `react-native-webview`, which wraps it on iOS) — Apple has required this over the deprecated `UIWebView` for years; App Review will reject apps using the old API.
- Give every WebView screen native chrome around it — a native nav bar with a title and back button, a loading state, and an explicit error state for the "ChurchSuite unavailable" case from §5.3 — so a ChurchSuite outage doesn't look like the *app* crashed.
- For the Give/Donate and event-payment WebViews specifically: since these are external payment flows under the nonprofit/physical-goods carve-outs in §6, HIG and App Review guidance both expect the flow to be clearly presented as leaving the app's own checkout context — a plain in-app WebView screen (not a disguised native-looking form) satisfies this.

B.3 Push notifications

- Request notification permission **contextually**, not on first launch — e.g. right after a member joins their first group, with a one-line explanation of what they'll be notified about. Cold, unexplained permission prompts on launch have a materially worse opt-in rate and read as generic rather than considered.
- Per §8/D4, notifications for chat should be **content-free** ("New message in your group") — this is also simply good HIG practice for lock-screen privacy, independent of the safeguarding rationale.
- Respect the user's **notification settings granularity** — Apple's guidance expects apps with multiple notification "types" (chat messages, event reminders, giving receipts) to let users control each category independently in-app, mirrored to iOS's per-app notification settings where practical.

B.4 Chat / messaging UI

- There is no dedicated Apple "Messaging" HIG page as a component, but the **Messages app's own patterns are the de facto reference** iOS users expect: grouped bubbles by sender, timestamps, a group name/member-avatar header, and a composer pinned above the keyboard with safe-area handling. Given there's no 1:1 pattern in this app (§4), the group header (member list, "leave group") is more prominent UI real estate than in a typical consumer chat app — worth designing deliberately rather than borrowing a DM-first layout.
- Since chat is not E2EE (§4.2.4) and is safeguarding-reviewable, the UI should **not** borrow visual language (lock icons, "end-to-end encrypted" badges) from apps like WhatsApp/Signal that would misrepresent the privacy model to members — HIG's broader principle of clarity/honesty in UI applies directly here, and it's also a straightforward safeguarding-transparency requirement from §4.5.

B.5 Onboarding & sign-in

- Because accounts are **provisioned against ChurchSuite records, not self-signed-up** (§3), the first-run flow is a "verify who you are" flow rather than a generic sign-up form — HIG's onboarding guidance favours getting users to value quickly with minimal upfront friction; here that means a short flow (email/invite-link verification) rather than a long profile-creation form, since ChurchSuite already holds the profile data.
- No phone number field anywhere in onboarding, consistent with §3/§4 — this also means skip Apple's SMS auto-fill/one-time-code affordances entirely, since there's no SMS step to autofill.

B.6 Audio/video playback

- **Video (sermons):** must use the official YouTube player surface per §7 — this already inherits Apple's expected playback controls (scrubber, fullscreen, AirPlay) via the YouTube SDK, so no custom playback-control design is needed there.

- **Audio (podcast):** a custom native player is worthwhile (§7). Follow the standard **Now Playing / lock-screen and Control Center integration** (`MPNowPlayingInfoCenter` under the hood of libraries like `expo-audio` / `react-native-track-player`) so playback controls, artwork, and scrubbing appear correctly outside the app — this is expected baseline behaviour for any audio app on iOS, not an optional nicety.

B.7 Accessibility

- **Dynamic Type:** all text (including inside chat bubbles and event/sermon cards) should scale with the user's chosen text size; avoid fixed-height containers that clip scaled text.
- **VoiceOver:** every icon-only control (tab bar icons aside, which get automatic labels from SF Symbols) needs an explicit accessibility label — particularly relevant for the chat composer, group member avatars, and audio player transport controls.
- **Sufficient touch targets** (44×44pt minimum) — relevant for chat message action affordances (redact/report) and player scrubbers, which are easy to design too small.

B.8 What to do with this appendix

Treat B.1–B.7 as a checklist to walk through with whoever does the actual visual/interaction design (Phase 1 onward), not as final decisions — at that stage, pull the live HIG pages for the specific components in play (tab bars, notifications, web content) since Apple does refine specifics over time even where the broad principles above are stable.